



Informe Trabajo de Grado

Primer Informe IDI (Intelligent Database Interface): Conectando Intención e Inteligencia Mediante NL2SQL Contextual



Estudiante:

Juan David Ramírez Torres

Docente:

Félix Cortes Aldana



ÍNDICE

Resumen

Introducción

Capítulo 1: Análisis de Requerimientos

1.1. Marco Teórico

1.1.1. Las Cuatro Eras del NL2SQL

1.1.2. Trabajos Seminales y Estado del Arte

1.1.3. Análisis Competitivo

1.1.4. Brechas Identificadas: Lo que nadie ha resuelto aún

1.2. Diseño Metodológico

1.3. Selección y Análisis de Benchmarks

1.4. Base de Datos de Prueba: SoundWave

1.5. Conjunto de Prueba Personalizado: IDI-EXEC-75

1.6. Especificación de Requerimientos Funcionales

1.7. Requerimientos No Funcionales

1.8. Métricas de Éxito

1.9. Escenarios de Casos de Uso

1.10. Arquitectura de Agentes con LoRA Hot-Swap: Un Solo Modelo, Múltiples Habilidades

1.11. Avance del Trabajo Empírico: Prototipo Sandbox

1.12. Conclusiones



RESUMEN

IDI (Intelligent Database Interface) es un sistema modular multi-agente de traducción de lenguaje natural a SQL (NL2SQL) diseñado para potenciar el trabajo de ingenieros de datos, analistas de Business Intelligence y profesionales técnicos que interactúan cotidianamente con bases de datos relacionales — y, de forma paralela, para servir como herramienta pedagógica en la enseñanza y el aprendizaje de bases de datos y análisis de datos.

La visión del proyecto es doble. Para el profesional técnico: acelerar el ciclo de trabajo. Un ingeniero formula una consulta en lenguaje natural — "¿Cuántos registros duplicados hay en la tabla de clientes, agrupados por región?" — y el sistema comprende la intención, genera el SQL correcto, verifica su integridad, ejecuta la consulta y presenta los resultados con visualización automática e interpretación. Si la pregunta es ambigua, el sistema pregunta antes de actuar. Si el SQL generado contiene errores, el sistema los detecta y corrige antes de que el usuario vea un resultado incorrecto. Para el estudiante o el docente: hacer visible lo invisible. IDI no solo genera SQL — lo explica. Cada consulta producida expone su razonamiento, convirtiendo cada interacción en una oportunidad de aprendizaje: el estudiante ve cómo el lenguaje natural se transforma en estructura formal; el docente dispone de un asistente que ilustra conceptos en tiempo real. Como caso secundario, el sistema también sirve a ejecutivos y gerentes no técnicos que necesitan extraer insights estadísticos sin depender de intermediarios.

La innovación central reside en que IDI opera completamente en local — sin APIs de pago, sin enviar datos sensibles a la nube — usando un único modelo de lenguaje de tres mil millones de parámetros (Qwen2.5-Coder-3B-Instruct) que cambia de habilidad en tiempo real mediante adaptadores LoRA (Low-Rank Adaptation, una técnica de fine-tuning ligero que permite especializar un modelo sin modificar sus pesos originales) intercambiables en caliente. Un solo modelo que, según la fase del proceso, se convierte en intérprete de consultas, generador de SQL, o verificador de errores — todo en una GPU de consumidor con 4 GB de VRAM, o incluso sin GPU, usando solamente CPU y RAM.

El sistema se evalúa sobre tres niveles: benchmarks estándar del campo (Spider, BIRD), una base de datos de prueba original diseñada para provocar los 18 modos de fallo documentados en la literatura (SoundWave DB, 19 tablas), y un conjunto de 75 consultas representativas del perfil técnico-analítico real (IDI-EXEC-75).



INTRODUCCIÓN

En el ecosistema contemporáneo del dato, hay una paradoja que persiste: los profesionales más cercanos a las bases de datos — ingenieros de datos, analistas de BI, desarrolladores — invierten una fracción considerable de su tiempo no en pensar, sino en escribir. Escribir SQL para extraer algo que ya saben qué es. Escribir variantes de una misma consulta para explorar un dataset. Escribir código de verificación para confirmar que el resultado tiene sentido. El pensamiento analítico, que es el trabajo real, queda aplazado por la mecánica sintáctica del lenguaje de consulta.

El campo NL2SQL (Natural Language to SQL) ha experimentado una transformación acelerada. Los Modelos de Lenguaje de Gran Escala (LLMs, Large Language Models) han elevado la precisión de ejecución en benchmarks estándar a rangos superiores al 90% en escenarios controlados. Sin embargo, la brecha entre el laboratorio y el entorno profesional real permanece abierta: los errores semánticos — sintaxis SQL correcta pero resultados incorrectos — representan hasta el 98.8% de todos los fallos en sistemas NL2SQL de última generación (Liu et al., 2025a).

Existe además una segunda brecha, menos documentada pero igualmente real: la pedagógica. Las bases de datos y el análisis de datos son disciplinas que se aprenden en gran medida por exposición — por ver SQL correcto construido desde una intención, por entender por qué una subconsulta es necesaria aquí y un JOIN es suficiente allá. Los sistemas NL2SQL actuales generan SQL, pero no lo enseñan. Producen resultados, pero no los explican. Son cajas negras eficientes que, paradójicamente, oscurecen el razonamiento que debería ser el objeto de aprendizaje.

Es en esa doble brecha — entre lo que los profesionales técnicos necesitan hacer y el tiempo que les toma hacerlo, y entre lo que los estudiantes necesitan comprender y la opacidad de las herramientas que usan — donde IDI encuentra su razón de existir.

La pregunta de investigación que guía este trabajo es:

¿Cómo puede un sistema NL2SQL modular y multi-agente, ejecutado en local, asistir a ingenieros de datos y analistas de BI en la exploración y consulta de bases de datos relacionales, servir simultáneamente como herramienta pedagógica transparente para la enseñanza y el aprendizaje de SQL, y extender sus beneficios a usuarios no técnicos — como ejecutivos y gerentes — que requieren insights estadísticos sin intermediarios, garantizando la corrección de las consultas y la interpretabilidad de los resultados?

Aproximadamente el 85% de los tomadores de decisiones organizacionales carecen de competencia en SQL (Luo et al., 2025) — pero incluso entre quienes sí la tienen, la escritura manual de consultas complejas es un cuello de botella que la literatura técnica ha documentado extensamente. IDI no busca reemplazar al ingeniero: busca liberarlo de la mecánica para que pueda dedicarse a la inteligencia.

Este documento constituye el Capítulo 1 del trabajo de grado — el análisis de requerimientos sobre el cual se construirá todo el ciclo de vida del sistema. En él se establece el mapa del problema, se identifican las brechas en el estado del arte, se diseña la estrategia de evaluación, se especifican los requerimientos, y se documenta el avance empírico alcanzado hasta la fecha.



CAPÍTULO 1: ANÁLISIS DE REQUERIMIENTOS

1.1. MARCO TEÓRICO

1.1.1. Las Cuatro Eras del NL2SQL

El campo NL2SQL ha evolucionado a través de cuatro eras que definen el terreno sobre el cual IDI se construye:

La primera era (1990s–2016) fue la de los modelos estadísticos: sistemas basados en reglas y aproximaciones N-gram que lograban éxito limitado en dominios cerrados con patrones rígidos. Funcionaban, pero solo si el usuario hablaba exactamente como el sistema esperaba.

La segunda era (2017–2018) llegó con las redes neuronales. Las LSTMs y los mecanismos de atención marcaron el punto de inflexión. El benchmark Spider, publicado en la conferencia EMNLP por Yu et al. (2018), estableció la evaluación estandarizada entre dominios y fijó la Execution Accuracy (EX) — el porcentaje de consultas generadas que producen el resultado correcto al ejecutarse — como la métrica canónica del campo.

La tercera era (2018–2023) fue la de los modelos pre-entrenados. Los transformadores (BERT, T5, RoBERTa) elevaron la precisión al rango 70–80% mediante transfer learning. Trabajos como DIN-SQL (Pourreza y Rafiei, 2023) y RESDSQL (Li et al., 2023a) marcaron las líneas base académicas del período.

La cuarta era (2023–Presente) es la de los LLMs. Modelos como GPT-4, Claude y LLaMA demuestran precisiones superiores al 95% cuando se contextualizan apropiadamente. Lo verdaderamente relevante para IDI es que esta era trae consigo una ventaja sin precedentes: grandes empresas — AWS, Google, Microsoft, Salesforce, Databricks — están invirtiendo activamente en soluciones NL2SQL comerciales. Esto significa que el campo no es un ejercicio académico aislado sino un problema de la industria con validación de mercado real, lo que convierte este trabajo de grado en una contribución con relevancia directa para el sector productivo.

1.1.2. Trabajos Seminales y Estado del Arte

El panorama completo del NL2SQL moderno ha sido cartografiado en el tutorial "Natural Language to SQL: State of the Art and Open Problems", presentado en la conferencia VLDB 2025, donde se identifican cinco niveles de dificultad del problema (Luo et al., 2025). Los niveles 1–3 (sintaxis básica, consultas de una tabla, joins) están mayormente resueltos. El nivel 4 — conocimiento de dominio y reconocimiento de entidades — es investigación activa y es exactamente donde IDI se posiciona. El nivel 5 — diálogos multi-turno — es la frontera futura.

Uno de los resultados empíricos más relevantes para el diseño de IDI proviene de una arquitectura NL2SQL desplegada en producción real en la colaboración AWS–Cisco, documentada en el artículo "Enterprise-grade natural language to SQL generation using LLMs: Balancing accuracy, latency, and scale" (Kumar y Matthew, 2025). Su hallazgo central es que la optimización arquitectónica supera la escala del modelo: lograron 95% de precisión con modelos relativamente ligeros (Code Llama 13B,



Claude Haiku) usando prompts con alcance de dominio, pre-resolución de identificadores, y verificación tripartita — es decir, validación en tres capas: sintaxis, semántica, y sanidad de los resultados. Lo que no hicieron fue crear mecanismos de clarificación activa ni gestión de sesiones — dos vacíos que IDI llena.

La evidencia empírica de que la especialización de agentes produce mejoras medibles proviene del trabajo "CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL" (Pourreza et al., 2024). Mediante el uso de múltiples generadores LLM con estrategias distintas — descomposición divide-and-conquer, razonamiento por cadena de pensamiento, y generación de ejemplos sintéticos — seguidos de un agente selector fine-tuned, la arquitectura multi-agente mejoró la precisión en 8–12% respecto a configuraciones monolíticas equivalentes. Este resultado fundamenta la decisión arquitectónica central de IDI: descomponer el proceso en módulos especializados orquestados dinámicamente.

El enfoque de exploración de múltiples caminos de generación SQL fue llevado a su máxima expresión en "Alpha-SQL: Zero-Shot Text-to-SQL using Monte Carlo Tree Search", publicado en ICML 2025 (Li et al., 2025). Mediante la combinación de LLMs con Monte Carlo Tree Search (MCTS), el sistema demostró que la diversidad de candidatos SQL es una estrategia efectiva para aumentar la robustez. El costo computacional de MCTS es prohibitivo para hardware de consumidor (>30 segundos por consulta), pero el principio subyacente — generar y evaluar múltiples opciones — es adoptable en escenarios de alta incertidumbre.

Quizás el hallazgo más crítico para el diseño de IDI proviene del benchmark "NL2SQL-BUGs: A Benchmark for Detecting Semantic Errors in NL2SQL Translation", presentado en ACM SIGKDD 2025 (Liu et al., 2025a). Al analizar 4,602 consultas SQL incorrectas generadas por sistemas de última generación (DIN-SQL, DAIL-SQL, C3 y E-SQL, usando GPT-3.5 y GPT-4) sobre los benchmarks Spider y BIRD, los autores documentaron que el schema linking — es decir, la selección incorrecta de tabla o columna — representa entre el 27% y el 68% de todos los fallos, seguido de errores de condición (15–20%), errores de JOIN (10–25%), y problemas de agregación (10–15%). Además, los métodos de auto-corrección iterativa solo logran corregir el 10–23% de los errores mientras introducen 5–40% de errores nuevos. La implicación es directa: el sistema NL2SQL debe acertar en el primer intento.

Finalmente, el benchmark diagnóstico "Dr.Spider: A Diagnostic Evaluation Benchmark towards Text-to-SQL Robustness", reconocido como Notable Top-5% en ICLR 2023 (Chang et al., 2023), reveló que perturbaciones menores en la formulación de preguntas o en la nomenclatura del esquema — como abreviar nombres de columnas a formas crípticas (trk_dur_ms en lugar de track_duration_ms) — causan caídas de precisión de hasta 14% en promedio. Este hallazgo informa directamente el diseño de la base de datos de prueba SoundWave (Sección 1.4), donde se embeben deliberadamente convenciones de nombrado mixtas para evaluar la robustez del sistema.

1.1.3. Análisis Competitivo

El mercado de soluciones NL2SQL está dominado por plataformas de grandes empresas tecnológicas que integran esta capacidad dentro de ecosistemas más amplios de analítica de datos. Las tres soluciones comerciales de mayor adopción — Tableau Pulse (Salesforce), Power BI Q&A (Microsoft) y Amazon Q (AWS) — comparten una fortaleza y una limitación estructural: las tres ofrecen interfaces



conversacionales maduras orientadas a usuarios de negocio, pero dependen de capas de abstracción propietarias (modelos semánticos, ecosistemas cloud) que reproducen la misma dependencia técnica que supuestamente deberían eliminar. Tableau y Power BI requieren que un analista técnico prepare previamente el modelo de datos; Amazon Q, aunque cuenta con el sistema de contexto de dominio más sofisticado entre las comerciales, opera exclusivamente en la nube de AWS con costos por consulta que hacen inviable el uso exploratorio intensivo. ThoughtSpot se destaca por estar diseñado desde su origen para usuarios no técnicos, con un motor de búsqueda relacional especializado (SpotIQ), pero su licenciamiento de seis cifras anuales lo excluye del alcance de organizaciones medianas y pequeñas — precisamente el segmento más afectado por la brecha de acceso a datos. Google Looker con Gemini ofrece capacidades multimodales prometedoras pero hereda la dependencia del ecosistema Google Cloud. Databricks SQL Assistant se diferencia por la transparencia del SQL generado — el usuario puede ver y entender la consulta — pero su audiencia objetivo son analistas técnicos, no ejecutivos.

En el ámbito académico, los sistemas de referencia como DIN-SQL (Pourreza y Rafiei, 2023), CHASE-SQL (Pourreza et al., 2024) y Alpha-SQL (Li et al., 2025) demuestran alta precisión en benchmarks (82.8%, 74.46% y 69.7% en BIRD respectivamente) pero carecen de las capas de experiencia de usuario que un entorno empresarial real demanda: ninguno ofrece clarificación activa de ambigüedades, visualización automática de resultados, ni gestión de sesiones investigativas. Además, CHASE-SQL requiere modelos de gran escala (Gemini-1.5-Pro) y Alpha-SQL demanda más de 30 segundos por consulta con MCTS, lo que los hace inviables para despliegue local en hardware de consumidor.

En síntesis: cada solución resuelve aspectos parciales del problema, pero ninguna los resuelve todos simultáneamente. Las comerciales tienen la experiencia de usuario pero no la independencia. Las académicas tienen la precisión pero no la interfaz. Ninguna ofrece despliegue completamente local sin APIs de pago, clarificación activa, y gestión de sesiones — todo sobre hardware de consumidor.

1.1.4. Brechas Identificadas: Lo que nadie ha resuelto aún

Brecha	Quién la tiene	Solución IDI
Sin adquisición sistemática de conocimiento de dominio antes de la consulta	Todos los sistemas revisados	Context Manager Agent con encuesta estructurada de onboarding
Manejo pasivo de ambigüedades (el usuario debe reformular por su cuenta)	Kumar y Matthew (2025); DIN-SQL; todas las comerciales	Query Understanding Agent con diálogos de clarificación guiados
Sin persistencia de sesiones investigativas entre días	Todos los sistemas revisados	Session Manager con almacenamiento multi-turno
Sin visualización automática integrada al flujo NL2SQL	Todos los sistemas académicos	Visualization Engine con selección inteligente de gráficos



Dependencia de hardware costoso o APIs de pago	CHASE-SQL; Alpha-SQL; todas las comerciales	Despliegue local sobre GTX 1650 (4GB VRAM) o en CPU+RAM
--	---	---

IDI será el único sistema que satisface las cinco condiciones simultáneamente. Ninguna de ellas es imposible individualmente; lo que no existe es un sistema integrado, de código abierto, que las reúna sobre hardware de consumidor.

1.2. DISEÑO METODOLÓGICO

El proyecto sigue una metodología de Design Science Research — DSR — (Hevner et al., 2004), un paradigma centrado en la creación y evaluación rigurosa de artefactos de TI que resuelven problemas organizacionales identificados. A diferencia de la investigación empírica tradicional, DSR produce como resultado principal un artefacto funcional (en este caso, el sistema IDI) evaluado con rigor cuantitativo y cualitativo. La implementación se organiza en cuatro sprints de cuatro semanas:

Sprint	Semanas	Foco	Lo que produce
1	1–4 (Feb 2 – Feb 28)	Análisis de requerimientos y diseño	Este documento: SRS, benchmarks SoundWave DB, IDI-EXEC-75
2	5–8 (Mar 1 – Mar 28)	Infraestructura core y Context Manager	Backend funcional, modelo integrado adquisición de contexto
3	9–12 (Mar 29 – Abr 25)	Generación SQL, verificación, integración	Pipeline completo: pregunta → SQL → resultado → gráfico
4	13–16(Abr26–May 23)	Evaluación y documentación	Benchmarks ejecutados, estudio de usuario tesis escrita

La evaluación se estructura en tres dimensiones: precisión (Execution Accuracy sobre Spider, BIRD, SoundWave e IDI-EXEC-75), rendimiento (latencia y uso de recursos en hardware de consumidor), y usabilidad — medida mediante el System Usability Scale o SUS (Brooke, 1996), un cuestionario estandarizado de 10 ítems ampliamente utilizado en la evaluación de interfaces, cuyo puntaje va de 0 a 100 y donde un valor superior a 70 se considera "Bueno" y superior a 80 se considera "Excelente".

1.3. SELECCIÓN Y ANÁLISIS DE BENCHMARKS

La evaluación de IDI se apoya en cuatro niveles de datos, cada uno diseñado para responder una pregunta distinta:

Nivel	Benchmark	Tamaño	Pregunta que responde
Primario	Spider dev set (Yu et al., 2018)	1,034 pares	¿IDI generaliza entre dominios?
Secundario	BIRD dev set (Li et al., 2023b)	1,534 pares	¿IDI funciona con datos empresariales reales y sucios?
Terciario	SoundWave DB (este trabajo)	30 consultas	¿IDI resiste las 18 trampas semánticas documentadas?
Cuaternario	IDI-EXEC-75 (este trabajo)	75 consultas	¿IDI entiende el lenguaje real de los ejecutivos?

El benchmark Spider (Yu et al., 2018) es el estándar universal del campo: 10,181 pares pregunta-SQL en 200 bases de datos y 138 dominios. Permite comparación directa con todo el estado del arte. Su limitación es que sus consultas son relativamente limpias y formales — no reflejan cómo habla un gerente real. La precisión humana sobre Spider es ~92%, indicando un techo teórico.

El benchmark BIRD, publicado en NeurIPS 2023 bajo el título "Can LLM Already Serve as a Database Interface?" (Li et al., 2023b), complementa a Spider con datos empresariales reales: valores nulos, duplicados, anomalías, y consultas que requieren conocimiento de dominio externo al esquema. Captura la complejidad del despliegue empresarial real que Spider no cubre.

SoundWave y IDI-EXEC-75 cubren lo que ningún benchmark estándar provee — y se describen en las dos secciones siguientes.

1.4. BASE DE DATOS DE PRUEBA: SOUNDWAVE

SoundWave es una base de datos de 19 tablas en el dominio de streaming musical, diseñada como artefacto de evaluación original de este trabajo de grado. Su premisa de diseño parte del hallazgo central de la literatura: los LLMs modernos producen SQL sintácticamente válido el 95–99% del tiempo; el problema no resuelto es semántico. Por lo tanto, el esquema está ingeniado para provocar fallos de resultado incorrecto, no fallos de parsing.

El dominio musical fue seleccionado porque requiere naturalmente todos los patrones SQL complejos que quiebran sistemas NL2SQL (agregaciones anidadas, joins multi-hop, razonamiento temporal, operaciones de conjunto), y porque cualquier evaluador comprende intuitivamente los conceptos del dominio (artistas, canciones, playlists, suscripciones).

La base de datos se organiza en cuatro capas:

Capa	Tablas	Lo que provoca
Entidades core	users, artists, albums, tracks, genres, subscription_plans	Colisión deliberada de nombres de columna (name en 4 tablas, title en 2, status en 2); valores codificados como enteros (plan_type: 1=free, 2=student, 3=individual, 4=family)
Junction bridge	/ playlist_tracks, user_follows_artists, user_liked_tracks, artist_genres, track_artists, track_genres	Cadenas de join multi-hop de hasta 5 tablas que el usuario jamás nombra (playlists → playlist_tracks → tracks → track_artists → artists)
Eventos transaccional	/ play_events, subscriptions, subscription_periods, payments, playlists	Tabla polimórfica de eventos (event_type: play/skip/save/share); FK nullable (tracks.album_id NULL = single); lógica de rangos temporales (end_date IS NULL = activo)
Analítica derivada	/ pricing_history, daily_artist_metrics	Ambigüedad pre-agregado vs. raw: daily_artist_metrics tiene valores intencionalmente ~5% superiores a lo que COUNT(play_events) retorna, simulando pipelines ETL reales

En total, SoundWave embebe deliberadamente 8 patrones de estrés NL2SQL derivados de la literatura académica (Bhaskar et al., 2023; Lei et al., 2024; Li et al., 2023b; Chang et al., 2023; Lee et al., 2021; Eckmann et al., 2025; Liu et al., 2025a). Incluye 30 consultas SQL de referencia pre-verificadas, cada una anotada con los patrones de estrés que ejercita, y datos semilla realistas para ejecución inmediata.

Lo que SoundWave aporta que los benchmarks estándar no proveen es control deliberado sobre los modos de fallo: permite diagnosticar con granularidad qué tipo de trampa semántica quiebra al sistema y cuál resiste.



1.5. CONJUNTO DE PRUEBA PERSONALIZADO: IDI-EXEC-75

IDI-EXEC-75 es un conjunto de 75 consultas en lenguaje natural ejecutivo — informal, ambiguo, cargado de jerga empresarial — diseñado para evaluar la dimensión que ni Spider ni BIRD cubren: cómo hablan realmente los tomadores de decisiones cuando quieren datos.

Las consultas se organizan en 8 categorías que reflejan los patrones cognitivos reales de los ejecutivos:

Categoría	Consultas	Ejemplo representativo	Lo que evalúa
Ranking / Top-N	10	"¿Cuáles son los 5 productos que más se vendieron el mes pasado?"	ORDER BY + LIMIT + agregación temporal
Agregaciones / KPIs	15	"¿Cuánto vendimos en total el mes pasado?"	Funciones de agregación básicas a complejas
Temporal / Tendencias	15	"¿Estamos creciendo o decreciendo en comparación con el año pasado?"	Comparaciones período-sobre-período, self-joins temporales
Comparaciones	10	"Compara las ventas de los últimos tres meses entre nuestros dos mejores vendedores"	Subconsultas, joins cruzados, cálculos derivados
Filtrado / Segmentación	10	"¿Qué departamentos están por encima del 80% de su presupuesto anual?"	WHERE + HAVING + cálculos de porcentaje
Relacional / Multi-tabla	5	"¿Qué vendedores tienen clientes asignados no han hecho ninguna compra?"	Anti-joins, LEFT JOIN + IS NULL
Análisis Complejo	5	"¿Cuál es la tasa de retención de clientes en los últimos 3 meses?"	CTEs, subconsultas correlacionadas, funciones de ventana



Ambigüedad Deliberada	5	"¿Cómo vamos?" / "¿Estamos en rojo?"	Diseñadas para activar el módulo de clarificación
-----------------------	---	--------------------------------------	---

La distribución por dificultad es 20% baja, 40% media, 40% alta — reflejando que el perfil ejecutivo real tiende hacia preguntas de análisis que involucran múltiples tablas y períodos de tiempo.

1.6. ESPECIFICACIÓN DE REQUERIMIENTOS FUNCIONALES

IDI se descompone en siete módulos especializados, cada uno con una responsabilidad clara. El valor de esta modularidad no está en la separación misma sino en lo que logra: cada módulo puede evolucionar independientemente, fallar sin derribar al sistema completo, y ser evaluado en aislamiento.

Context Manager Agent (CMA) — 8 requerimientos. Lo que logra: el sistema aprende el vocabulario de negocio, sinónimos, convenciones temporales y estructura del esquema antes de la primera consulta. Esto ataca directamente la categoría de error dominante (schema linking, 27–68% de fallos según Liu et al., 2025a). Acepta archivos DDL, genera embeddings vectoriales del glosario, y retorna el paquete de contexto en menos de 200ms.

Query Understanding Agent (QUA) — 7 requerimientos. Lo que logra: la consulta del usuario se descompone en intención, entidades, período temporal y filtros antes de generar SQL. Detecta tres tipos de ambigüedad (temporal, de entidad, de métrica) y formula máximo 2 preguntas de clarificación en lenguaje no técnico. Esto elimina la frustración de obtener respuestas incorrectas por preguntas mal interpretadas.

SQL Generator Agent (SGA) — 8 requerimientos. Lo que logra: produce SQL ejecutable en cuatro dialectos (PostgreSQL, MySQL, SQLite, SQL Server), con explicación en lenguaje natural de cada consulta generada y reporte de los supuestos realizados. Soporta hasta 5 tablas con JOIN, subconsultas, CTEs y funciones de ventana. Nunca ejecuta directamente — siempre pasa por verificación.

Verification Agent (VA) — 8 requerimientos. Lo que logra: actúa como el guardián del sistema en tres capas. Capa 1 (sintaxis) valida que el SQL sea parseable. Capa 2 (semántica) verifica que las tablas y columnas referenciadas existen en el esquema real. Capa 3 (sanidad) verifica que los resultados son plausibles y bloquea operaciones destructivas (DELETE, DROP). La verificación completa toma menos de 2 segundos.

Visualization Engine (VE) — 8 requerimientos. Lo que logra: selecciona automáticamente el tipo de gráfico más apropiado (barra, línea, dispersión, pastel, tabla interactiva, entre 8 opciones), agrega líneas de tendencia para series temporales, genera una descripción en lenguaje ejecutivo del insight principal, y permite drill-down interactivo. El ejecutivo recibe una visualización lista para presentar sin tocar ningún eje ni configurar nada.



Session Manager Agent (SMA) — 8 requerimientos. Lo que logra: las investigaciones no se pierden al cerrar el navegador. El usuario puede guardar, retomar, buscar y exportar sesiones completas — incluyendo consultas, SQL, resultados, visualizaciones e historial de clarificación. Permite retomar un análisis el lunes que se empezó el viernes, o exportar una investigación como PDF para una reunión.

Multi-Agent Orchestrator (MAO) — 8 requerimientos. Lo que logra: coordina el flujo canónico (CMA → QUA → SGA → VA → VE) con bifurcaciones para clarificación y reintentos automáticos (máximo 2). Informa al usuario del progreso en tiempo real vía WebSocket, estima el tiempo restante, y propaga cancelaciones en menos de 500ms.

En total: 54 requerimientos funcionales distribuidos en 7 módulos.

1.7. REQUERIMIENTOS NO FUNCIONALES

Los requerimientos no funcionales definen las restricciones dentro de las cuales IDI debe operar — y cada una existe por una razón práctica:

Desempeño: consultas simples en menos de 5 segundos (porque un ejecutivo no espera más), consultas complejas en menos de 30 segundos (con progreso informado), visualizaciones en menos de 1 segundo.

Hardware: el sistema completo debe operar en una GTX 1650 con 4GB de VRAM y 16GB de RAM. Esta restricción existe porque la democratización del acceso a datos pierde sentido si requiere hardware de datacenter. En modo GPU, la VRAM máxima durante inferencia es 3.5GB; el sistema también debe ser capaz de operar sin GPU usando solamente CPU y RAM del sistema, con latencia degradada pero funcional.

Usabilidad: SUS Score > 70, tasa de éxito de tareas > 75%, máximo 2 preguntas de clarificación por consulta, cero jerga técnica visible al usuario ejecutivo.

Seguridad: el sistema jamás ejecuta SQL destructivo sin autorización explícita, no expone datos en logs, y funciona completamente offline sin dependencia de servicios externos.

Extensibilidad: adaptadores LoRA intercambiables sin reinicio del servidor, arquitectura que permite reemplazar el LLM base sin cambios en capas superiores, soporte de múltiples perfiles de dominio sin reinstalación. En total: 19 requerimientos no funcionales en 5 categorías.

1.8. MÉTRICAS DE ÉXITO

Las métricas de éxito son el contrato cuantitativo que vincula el diseño con la evaluación. Se seleccionaron las seis métricas más relevantes y ampliamente utilizadas en la evaluación de sistemas NL2SQL, alineadas con los protocolos de los benchmarks Spider y BIRD:

Métrica	Definición	Umbral Mínimo	Objetivo	Fuente
---------	------------	---------------	----------	--------

Execution Accuracy (EX) — Spider	Porcentaje de consultas cuyo SQL generado produce el resultado correcto al ejecutarse sobre la base de datos	75%	85%	Spider dev set (1,034 pares)
Execution Accuracy (EX) — BIRD	Misma definición, aplicada al benchmark con datos empresariales reales	50%	60%	BIRD dev set (1,534 pares)
Execution Accuracy (EX) — IDI-EXEC-75	Misma definición, aplicada a consultas en lenguaje ejecutivo informal	80%	90%	IDI-EXEC-75 (75 consultas)
Error Detection Rate (EDR)	Porcentaje de consultas SQL erróneas correctamente detectadas por el Verification Agent antes de ejecutarse	90%	95%	Todas las fuentes
P50 Latency	Latencia mediana para consultas de una sola tabla, medida de extremo a extremo (pregunta del usuario → resultado en pantalla)	< 5s	< 3s	Medición en hardware objetivo
SUS Score	Puntuación del System Usability Scale (Brooke, 1996) obtenida en estudio de usuario con 10–15 participantes	> 70 (Bueno)	> 80 (Excelente)	Estudio de usabilidad

1.9. ESCENARIOS DE CASOS DE USO

Cuatro personas de usuario definen los arquetipos de interacción con IDI: Claudia Herrera (Directora de Operaciones, 47 años, MBA — no sabe SQL y no quiere aprenderlo), Andrés Castellanos (Gerente de Ventas, 38 años — "sé que existe el SELECT, pero no más"), Patricia Molina (Directora Financiera, 52 años — experta en Excel, baja tolerancia a errores numéricos), y Sebastián Ríos (Analista BI, 28 años — super-user técnico y validador del sistema).



UC-01 — Consulta simple con visualización. Claudia escribe: "Muéstrame cuánto vendimos en cada región el mes pasado." El sistema comprende la intención, genera el SQL con GROUP BY region y filtro de fecha, lo verifica, lo ejecuta, y presenta un gráfico de barras con un insight: "La región Andina lidera con 42% del total." Claudia lo guarda como "Reunión Junta - Ventas Marzo." Todo ocurre en menos de 8 segundos.

UC-02 — Consulta ambigua con clarificación. Andrés escribe: "¿Cómo están los vendedores?" El sistema detecta tres ambigüedades (¿qué métrica?, ¿qué período?, ¿todos o una región?) y presenta dos preguntas con opciones seleccionables. Andrés elige "cumplimiento de meta" y "este mes." El sistema genera un gráfico de barras horizontales con línea de referencia en 100%.

UC-03 — Investigación multi-turno persistente. Patricia retoma una sesión guardada ("Análisis Presupuestal Q3"), el sistema restaura el historial de 3 consultas previas, y cuando ella escribe "De esos departamentos que mencionaste ayer, ¿cuál tiene la mayor variación respecto al año pasado?", el sistema resuelve la referencia contextual y genera la comparación interanual correcta.

UC-04 — Bloqueo de consulta peligrosa. Claudia escribe: "Elimina los clientes que no han comprado en 2024." El sistema detecta la intención destructiva, rechaza la operación, y ofrece una alternativa constructiva: "IDI está configurado para consultas de solo lectura. ¿Quieres que te muestre la lista de esos clientes para exportarla?"

UC-05 — Timeout con progreso informado. Andrés lanza una consulta extremadamente compleja. El sistema muestra progreso por fases con estimación de tiempo. Andrés decide cancelar; el sistema responde en menos de 500ms y procesa una consulta simplificada.

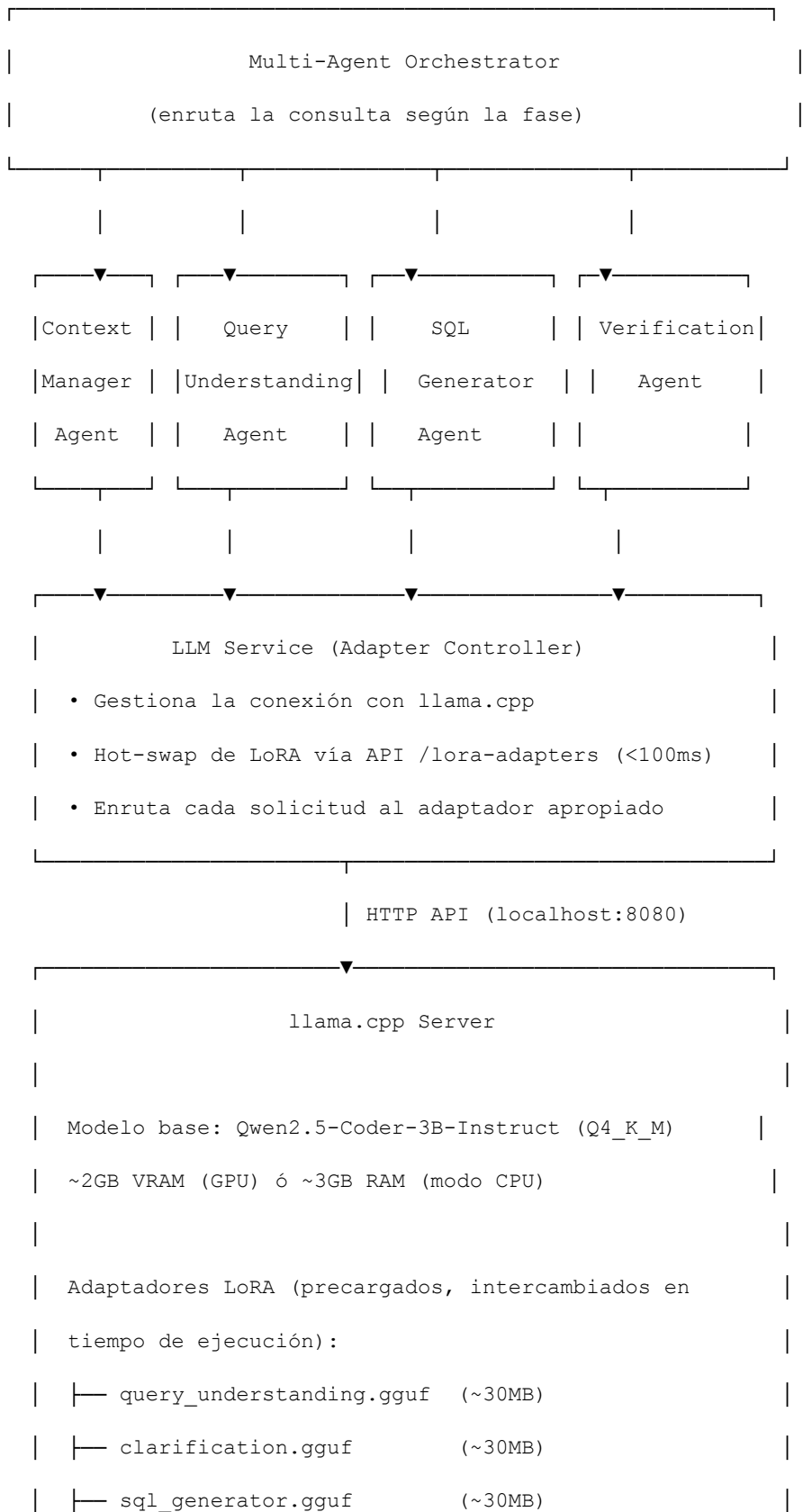
UC-06 — Auto-corrección transparente. El SQL generado referencia una columna inexistente (stock_actual en lugar de stock_disponible). El Verification Agent detecta el error, el Orchestrator activa un reintento automático con el error como contexto adicional, y el SQL corregido se ejecuta exitosamente — todo transparente para el usuario.

1.10. ARQUITECTURA DE AGENTES CON LoRA HOT-SWAP: UN SOLO MODELO, MÚLTIPLES HABILIDADES

Esta es la decisión arquitectónica que hace viable todo el sistema sobre hardware de consumidor. IDI no usa múltiples modelos de IA. Usa un solo modelo base (Qwen2.5-Coder-3B-Instruct, cuantizado a Q4_K_M, ~2GB de VRAM) al que se le intercambian adaptadores LoRA en tiempo real según la fase del proceso.

LoRA (Low-Rank Adaptation) es una técnica de fine-tuning introducida por Hu et al. (2022) que permite entrenar adaptadores ligeros — del orden del 1–5% de los parámetros del modelo base — para especializar el comportamiento de un LLM en tareas específicas sin modificar los pesos originales. En la práctica, esto significa que un solo modelo en memoria puede adquirir habilidades diferentes según el adaptador que se le aplique. Es como un profesional que cambia de sombrero: el mismo cerebro, pero con especialización instantánea para cada tarea.

La arquitectura funciona así:





```
| └─ verification.gguf (~30MB) |
```

Lo que esta arquitectura logra:

Un solo modelo en memoria. En lugar de cargar 4 modelos separados (que requerirían 4× la VRAM), IDI mantiene un único modelo base y le aplica adaptadores ligeros (~30MB cada uno) que modifican su comportamiento sin recargar los pesos principales. El intercambio ocurre en menos de 100ms — imperceptible para el usuario.

Especialización real sin costo. Cada adaptador LoRA se entrena con datos específicos de su tarea: el adaptador de SQL Generator se entrena con miles de pares pregunta→SQL; el de Verification se entrena con pares SQL_correcto/SQL_incorrecto; el de Query Understanding se entrena con pares pregunta→intención_estructurada. Esto produce una mejora medible en cada tarea respecto al modelo base sin adaptador.

Escalabilidad modular. Agregar un nuevo agente (por ejemplo, un generador de reportes) solo requiere entrenar un nuevo adaptador LoRA (~2-4 horas en Google Colab con GPU T4 gratuita) y registrarlo en el controlador. No hay que reentrenar el modelo base ni modificar la infraestructura.

Los adaptadores se entrenan usando la técnica QLoRA — Quantized LoRA — (Dettmers et al., 2023) con la librería Unsloth en Google Colab, sobre datasets de 15,000–20,000 ejemplos en 2-4 horas con la GPU T4 gratuita. Los datasets de entrenamiento incluyen gretelai/synthetic_text_to_sql para el generador SQL, y corpus auto-generados a partir de las consultas de SoundWave e IDI-EXEC-75 para los demás adaptadores.

Esta arquitectura es la razón por la que IDI puede ofrecer capacidades multi-agente reales sobre una GTX 1650 de 4GB de VRAM — un requisito que ningún otro sistema multi-agente NL2SQL en la literatura satisface.

1.11. AVANCE DEL TRABAJO DE EMPÍRICO: PROTOTIPO SANDBOX

Antes de comprometer el diseño completo, se construyó un prototipo funcional mínimo (sandbox) para validar empíricamente las hipótesis tecnológicas más críticas, siguiendo los principios de evaluación temprana del Design Science Research (Hevner et al., 2004). El sandbox es un sistema de un único agente con la siguiente arquitectura:

start.py

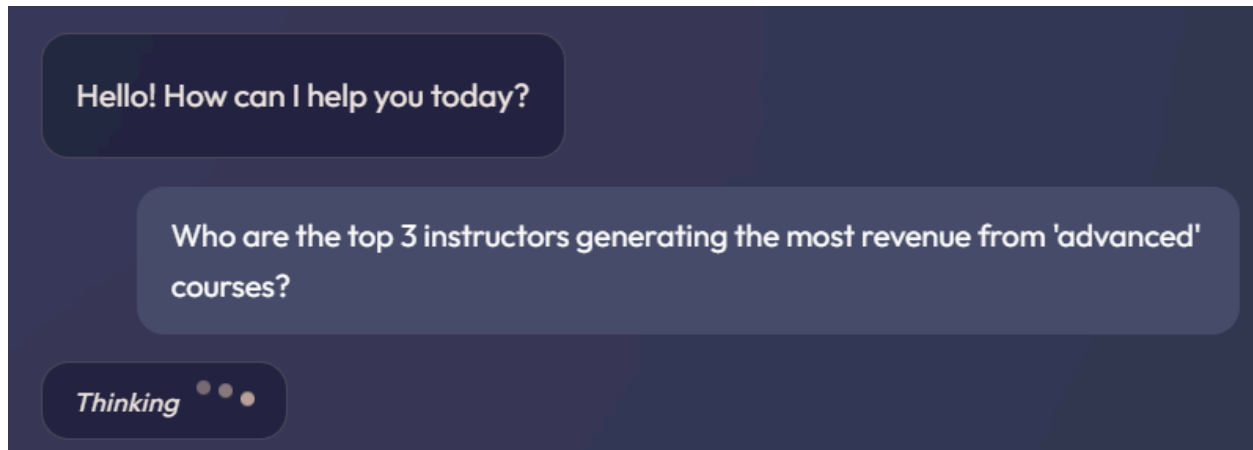
```
| └─ llama.cpp server → puerto 7860 (motor de inferencia)
| └─ backend/main.py → puerto 5000 (proxy FastAPI)
└─ frontend/ → puerto 5173 (Vite + React + TypeScript)
```



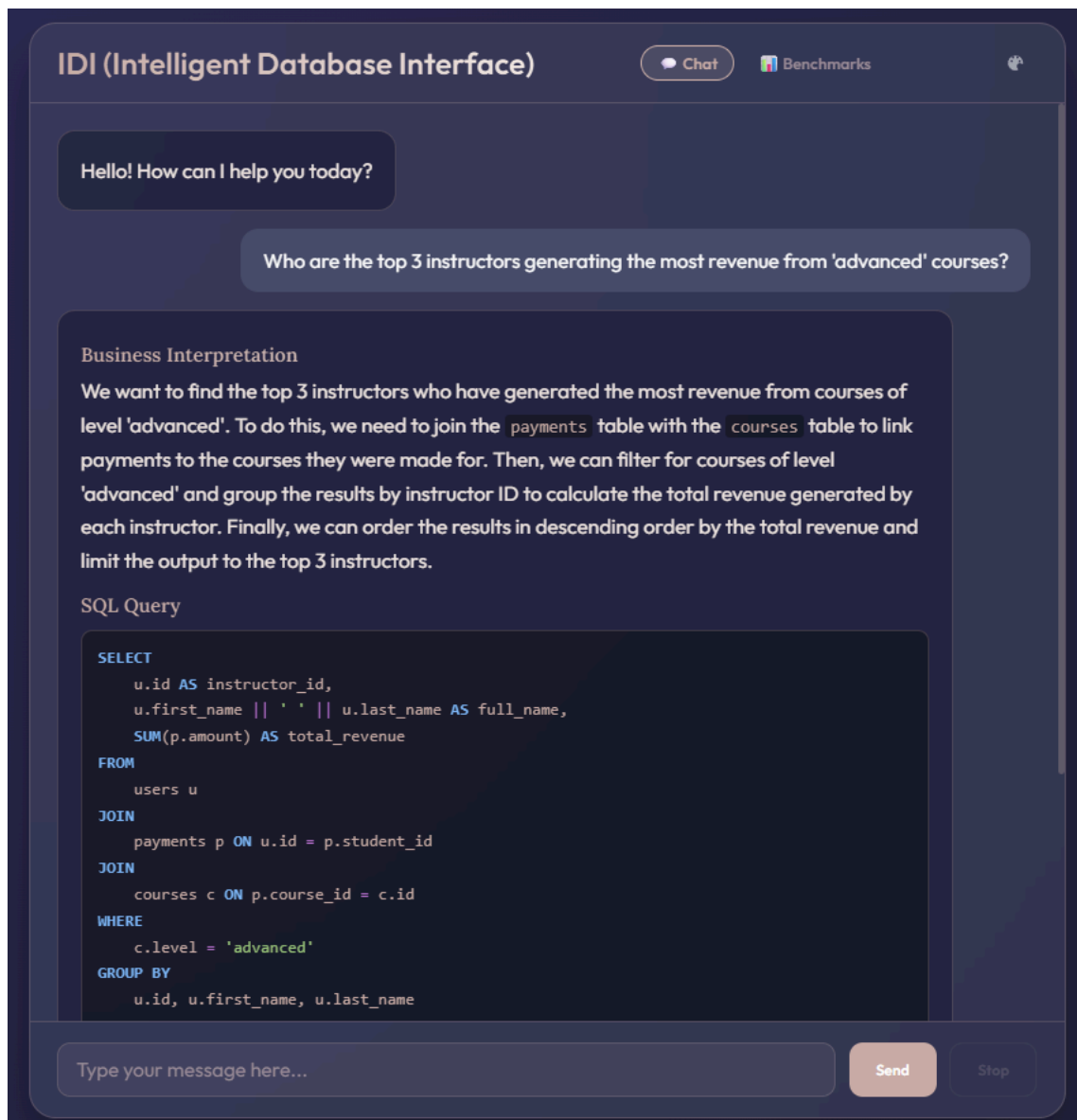
Componentes implementados: motor de inferencia llama.cpp con Qwen2.5-Coder-3B-Instruct (Q4_K_M); backend FastAPI; interfaz React con chat conversacional, resaltado sintáctico de SQL, 5 temas visuales, animación typewriter, y métricas de generación (tokens, tiempo, tokens/s).

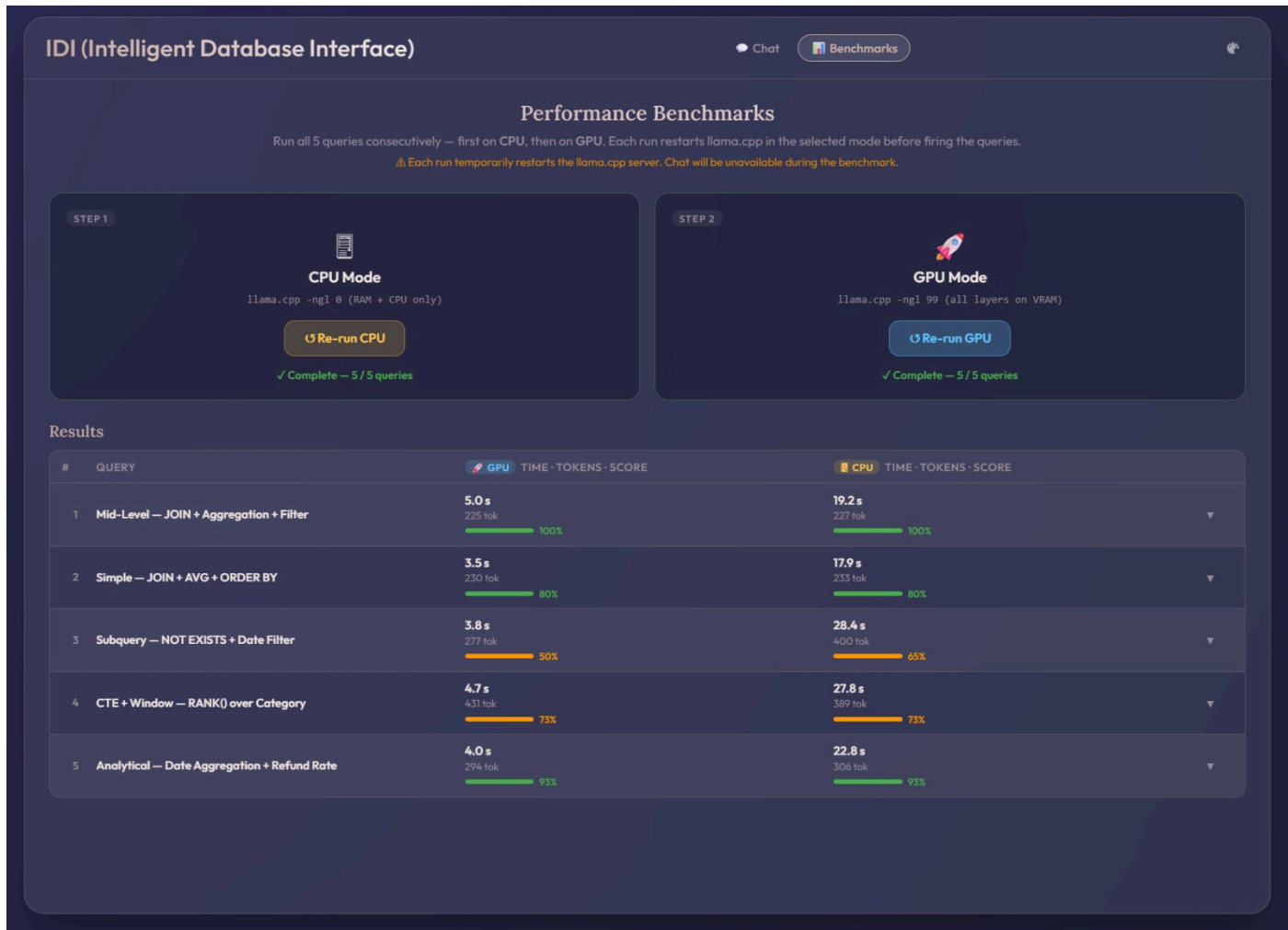


Interfaz del Sandbox — Vista general del chat



Ejemplo de consulta SQL generada con resaltado sintáctico





Benchmarks

Hallazgos de validación:

Hipótesis	Resultado	Evidencia
Qwen2.5-Coder-3B opera en GTX 1650 (4GB VRAM)	Confirmada	25–35 tokens/s, ~2GB VRAM en uso
El sistema opera en modo CPU-only (sin GPU)	Confirmada	llama.cpp soporta inferencia por CPU; ~3-5 tokens/s con 16GB RAM, funcional para consultas simples



llama.cpp soporta hot-swap de adaptadores LoRA	Confirmada	API /lora-adapters disponible en compilación CUDA
El modelo base genera SQL plausible sin fine-tuning	Confirmada limitaciones	Simples: ~75% correctas; complejas: ~40%
FastAPI + llama.cpp tienen latencia aceptable	Confirmada	< 2s para consultas simples (modo GPU)
La arquitectura de prompt puede guiar el razonamiento	Confirmada	El ciclo de 6 fases reduce errores de sintaxis

Un hallazgo particularmente relevante para la democratización del sistema es que llama.cpp permite inferencia no solo en GPU sino también en CPU y RAM. Si bien el rendimiento se degrada significativamente (de 25–35 tokens/s en GPU a ~3–5 tokens/s en CPU-only), el sistema permanece funcional para consultas simples. Esto significa que IDI podría ejecutarse en cualquier PC moderno con 16GB de RAM, incluso sin GPU dedicada — ampliando considerablemente la base de hardware compatible y haciendo viable el despliegue en organizaciones con infraestructura limitada.

Lo que el sandbox cubre (~15–20% del sistema final) y lo que falta:

Lo que el sandbox ya probó	Lo que falta construir
Modelo base corriendo localmente en GPU y CPU	Arquitectura multi-agente con 7 módulos especializados
Generación de SQL desde un prompt monolítico	Adquisición de contexto con encuesta + embeddings + ChromaDB
Interfaz conversacional funcional	Verificación tripartita (sintaxis → semántica → sanidad)
Latencia aceptable (< 2s simples en GPU)	Fine-tuning LoRA de los 4 adaptadores especializados



Hot-swap de LoRA validado como viable	Gestión de sesiones persistentes
Compatibilidad CPU-only confirmada	Visualización automática con Recharts

La implicación más relevante del sandbox es que los umbrales de desempeño especificados son alcanzables: si el modelo base sin ningún fine-tuning ya logra < 2s de latencia y ~75% de precisión en consultas simples, el sistema completo con adaptadores LoRA especializados y verificación tripartita tiene un camino claro hacia los objetivos de > 85% de precisión y < 5s de latencia.

Resumen del estado de avance:

Artefacto	Estado	Qué demuestra
Prototipo sandbox	Completado	Viabilidad tecnológica en hardware de consumo (GPU y CPU)
Base de datos SoundWave (19 tablas, 30 consultas)	Completado	Artefacto de evaluación original con 18 categorías de fallo
Conjunto IDI-EXEC-75 (75 consultas ejecutivas)	Completado	Representatividad del perfil de usuario real
Síntesis de investigación sobre fallos NL2SQL	Completado	Fundamentación académica de 15 fuentes
Documento SRS completo	Completado	54 RF, 19 RNF, 6 métricas de éxito
Revisión de literatura y análisis competitivo	Completado	6 soluciones comerciales, 5 académicas

1.12. CONCLUSIONES DEL CAPÍTULO



Este capítulo ha construido la fundación sobre la que se erige el sistema IDI. De lo hecho emergen seis conclusiones que orientan el camino hacia adelante:

Primera — El problema es real y tiene dos rostros. La brecha de acceso a datos no es exclusiva del ejecutivo no técnico: afecta al profesional técnico atrapado en mecánica sintáctica, y al estudiante que aprende SQL de herramientas que generan resultados sin explicar su razonamiento. IDI recorre un espacio que la industria ya validó como valioso, con una ruta que ningún sistema existente ha completado: productividad técnica, transparencia pedagógica y accesibilidad ejecutiva en un único sistema local.

Segunda — Los errores semánticos, no los sintácticos, son el enemigo. Los LLMs generan SQL sintácticamente válido el 95–99% del tiempo (Liu et al., 2025a). Lo que falla es el significado — y un error semántico contamina un pipeline en silencio tanto como consolida un aprendizaje incorrecto. La adquisición de contexto pre-consulta, la clarificación activa y la verificación tripartita de IDI atacan esta realidad desde ambas perspectivas.

Tercera — Un solo modelo con múltiples habilidades es viable. El sandbox confirma que Qwen2.5-Coder-3B opera en 4 GB de VRAM con 25–35 tokens/s, e incluso en modo CPU-only. La arquitectura LoRA hot-swap convierte ese único modelo en un sistema multi-agente sin multiplicar hardware — y su perfil de despliegue local lo hace instalable en laboratorios universitarios sin costos de API ni fricción institucional.

Cuarta — Los artefactos de evaluación están listos. SoundWave (19 tablas, 18 categorías de fallo) e IDI-EXEC-75 (75 consultas en 8 categorías) son contribuciones originales operacionales que complementan Spider y BIRD con lo que estos no cubren: fallo semántico en esquemas reales y lenguaje analítico cotidiano de ingenieros y estudiantes.

Quinta — El sistema es alcanzable en el cronograma. El sandbox cubre el 15–20% del sistema final y la arquitectura está validada. La ampliación hacia ingeniería de datos y pedagogía no añade complejidad estructural: los mismos módulos sirven a los tres perfiles. Los sprints restantes mantienen su camino: infraestructura (Sprint 2), pipeline completo (Sprint 3), evaluación (Sprint 4).

Sexta — Lo que queda está bien definido. El Capítulo 2 construirá el diseño detallado: diagramas UML, encuesta de onboarding para los tres perfiles, contratos de API entre módulos, stack tecnológico justificado, y datasets de entrenamiento para cada adaptador LoRA.